Experiment Report: Compilers Stage 2

罗兰 2021080066

Introduction

This report outlines the work completed for Stage 2 of the compiler implementation for MiniDecaf. In this stage, we focus on expanding our compiler's abilities to handle local variable definitions, assignments, and basic arithmetic operations, while also introducing a symbol table, semantic analysis, intermediate code generation, and simple register allocation for target code generation. The example used throughout this experiment is a basic C program that initializes a variable and returns its value.

```
int main() {
    int x = 2024;
    return x;
}
```

Lexical and Syntax Analysis

For this stage, we enhanced our Abstract Syntax Tree (AST) to represent the definitions and assignments of local variables. Specifically, new nodes have been added:

- `TInt` : Represents an integer type.
- `Identifier` : Represents variable identifiers (e.g., variable names).
- `Assignment` : Represents assignment operations, extending the `Binary` class.
- `Declaration` : Represents variable declarations, containing type (`var_t`), identifier (`ident`), and initial expression (`init_expr`).

Assignment operations are treated as a specialized binary operation, allowing reuse of existing binary operation logic.

Semantic Analysis

Semantic analysis checks the validity of variable definitions and usage. In this step, we utilize a symbol table to track variable declarations and enforce scoping rules. We implemented semantic analysis to:

1. Add new variable declarations to the symbol table.
2. Prevent redefinition of variables within the same scope.
3. Validate that variables are declared before being used.

For instance, if we modify the example program to use an undeclared variable `y` :

```
int main() {
    int x = 2024;
    return x + y;
}
```

The semantic analysis would report an error when attempting to use `y` since it is not declared. The symbol table is implemented using a `Scope` class, and we manage a single symbol table for the `main` function's scope.

Intermediate Code Generation

For the intermediate code, we generate Three Address Code (TAC) to represent the example program. The TAC output for `int x = 2024;` is as follows:

```
main:
    _T1 = 2024
    _T0 = _T1
    return _T0
```

The `ASSIGN` TAC instruction was used to represent the assignment operation (`_T0 = _T1`). In this stage, we continued using virtual registers (e.g., `_T1` , `_T0`) to store values, mapping MiniDecaf variables to these virtual registers.

Target Code Generation

The target code for the example program is generated in RISC-V assembly language. The generated RISC-V code is:

```
.text
.global main
main:
    li t1, 2024
    mv t0, t1     # Translate ASSIGN instruction
    mv a0, t0
    ret
```

We use RISC-V instructions such as `li` (load immediate) and `mv` (move) to translate the intermediate representation. Here, `t1` and `t0` represent RISC-V registers used to store the intermediate values.

Register Allocation and Stack Frames

Since the number of virtual registers often exceeds the number of physical registers available in the target architecture, we introduced a heuristic-based register allocation algorithm. This algorithm is based on liveness analysis and uses a simple heuristic to decide whether a temporary variable should be spilled to the stack frame.

In this stage, the implemented algorithm handles the allocation as follows:

1. Checks if a free register is available.
2. If all registers are occupied, spills the least recently used temporary variable to the stack.

We also reviewed the concept of a stack frame, which includes temporary variable storage and helps manage function calls. In the current stage, we only consider the stack frame for the `main` function.

Example Stack Frame

In a sample function like `calculate()` that defines multiple local variables:

```
int calculate() {
    int a = 1, b = 2, c = 3, d = 4, e = 5;
    int result = a + b + c + d + e;
    return result;
}
```

Assuming only four registers are available, one variable (`e`) would be stored in the stack frame. We provided assembly code for managing stack space accordingly.

Code Diff Summary

In this experiment, we updated several key components:

- Enhanced `riscvasmemitter.py` to add new `visitAssign` and `visitBinary` methods for handling assignment and binary operations.
- Implemented necessary changes in `tacgen.py` for handling variable declarations and assignments during TAC generation.
- Updated `namer.py` to declare variables in the symbol table and handle symbol lookup.
- Added new operations (e.g., `REM` , `EQU` , `NEQ` , `LEQ` , `GEQ`) to `tacop.py` to support additional binary operations.

Thought Question

Thought Question 1: Expanding Stack Frame Space in RISC-V

Assuming the stack pointer (`sp`) holds the current top address of the stack frame, the RISC-V assembly code to expand the stack frame by 16 bytes would be:

```
addi sp, sp, -16
```

This instruction subtracts 16 from the stack pointer, thereby expanding the stack frame space by 16 bytes. Note that the stack in RISC-V grows from high addresses to low addresses.

Thought Question 2: Handling Multiple Variable Definitions in the Same Scope

If MiniDecaf allows multiple definitions of the same variable name in the same scope (similar to Rust), with the new definition shadowing the previous one, the logic for variable definition and lookup must be adjusted.

1. **Definition Logic**: When defining a new variable with the same name, instead of preventing redefinition, the symbol table should add the new definition, shadowing the previous one. This could be achieved by maintaining a stack of symbols for each variable name, where the most recent definition is at the top of the stack.
2. **Lookup Logic**: When looking up a variable, the symbol table should return the most recent definition by accessing the top of the stack for that variable name. When leaving the scope, the top definition should be popped from the stack, effectively unshadowing the previous definition.

Summary

Step 5 primarily covered the introduction of the symbol table, register allocation, and stack frame concepts, presenting some challenges that required an understanding of scoping, memory management, and assembly-level optimizations. It is recommended to start early and build incrementally to overcome the complexity of this stage.